CS633 Assignment 2

Group 52

Vihaan Sapra (231149), Nischay Agarwal (230705), Yug Dharajiya (230362), Pallav Rastogi (230731), Aryamann Srivastava (230211)

Contribution: Equal contribution by all members.

1 Code Description

The code implements a 3D domain decomposition over `px` × `py` × `pz` MPI processes. Each process owns a local sub domain of size `nx` × `ny` × `nz` with `F` scalar fields. Over `T` time steps, the program performs halo exchange, stencil averaging and isovalue counting.

The code is organized into the following six logical segments:

1. MPI initialization and argument parsing
2. Data layout, memory allocation, and MPI datatype creation
3. Iterative halo communication and stencil computation loop
4. Isovalue counting (inner and outer)
5. Global reduction and output
6. Cleanup and finalization

1.1 MPI Initialization and Argument Parsing

Each process initializes MPI and parses 12 command-line arguments: `d, ppn, px, py, pz, nx, ny, nz, T, seed, F, isovalue`. The stencil radius is derived as $r = (d-1)/6$, which requires **d mod 6** to be equal to 1. Halo-extended array dimensions are set as $NX = nx + 2r$, $NY = ny + 2r$, $NZ = nz + 2r$.

1.2 Data Layout and Memory Allocation

Data is stored as a single flat 1D array using ZYX ordering with field-interleaved layout:

```
data[field+F*IDX(x, y, z)]    // IDX(a,b,c) = a*NY*NZ + b*NZ + c
```

This layout keeps all fields at the same spatial point contiguous in memory, improving cache utilization during stencil computation. Two arrays (`data` and `new_data`) are allocated to allow in place double-buffered updates.

Two custom MPI datatypes are created for non-contiguous halo faces:

- `faceY`: `MPI_Type_vector` spanning r layers in the Y direction across all X and Z.
- `faceZ`: `MPI_Type_vector` spanning r layers in the Z direction across all X and Y.

X-face halos are contiguous in memory and use a plain `MPI_DOUBLE` count.

1.3 Halo Exchange (`transferData`)

Non-blocking `MPI_Irecv` and `MPI_Isend` calls are issued for all 6 faces simultaneously. Receives are posted before sends to avoid potential deadlock. Tags are assigned to distinguish the two communication directions per axis. A single `MPI_Waitall` synchronizes all 12 requests (6 send and 6 receive requests, 1 per face) before boundary stencil computation.

1.4 Stencil Computation

The d-point stencil averages a cell with its 6 face neighbors (for $d = 7$, $r = 1$):

$$\mathrm{val}(P,\, t+1) = \frac{\sum_{k \in \{P,E,W,N,S,F,B\}} \mathrm{val}(k,\, t)}{\text{count of valid neighbors}}$$

Computation is split into two phases to overlap with communication:

- `avgInnerData`: Processes interior cells that do not depend on halo data. Called while halo exchange is happening.
- `avgOuterData`: Processes the outer shell of cells that depend on received halo data. Called after `MPI_Waitall`. Also swaps `data` and `new_data` pointers.

At domain boundaries (no neighbor), the denominator is reduced to reflect only the available neighbors.

1.5 Isovalue Counting

Isovalue counting uses the following method: a crossing is detected between adjacent grid points v and v' if $(v < \mathrm{iso} < v')$ or $(v' < \mathrm{iso} < v)$. Only positive direction neighbors (East, North, Up) are checked per point to avoid double counting within a process. Counting is also split:

- `countInnerIsoValue`: Counts crossings for interior cells (no boundary edges). Called while the next halo exchange is happening.
- `countOuterIsoValue`: Counts crossings for the outer shell, which may involve halo data. Called after `MPI_Waitall`. Finishes with `MPI_Reduce` to sum `local_count` across all processes into `count` on rank 0.

1.6 Main Loop and Communication/Computation Overlap

The loop is designed to overlap asynchronous communication with computation. An initial halo exchange is posted before the loop, while inner stencil averaging is performed concurrently. A call to `MPI_Waitall` ensures halo completion before the first iteration.

In each iteration, the outer shell stencil is computed using halo data, followed by swapping **data** and **new_data**. The next halo exchange is then posted immediately. During this communication, two independent tasks-**inner stencil averaging** (skipped in the final iteration) and **inner isovalue counting**-are performed on interior cells. After **MPI_Waitall**, outer shell isovalue counting is completed, and **MPI_Reduce** aggregates results to rank 0.

This design hides halo communication latency behind inner computation, leaving only outer shell work and `MPI_Reduce` on the critical path.

2 Code Optimizations

1. **Non blocking communication**: `MPI_Irecv`/`MPI_Isend` are used instead of blocking calls, enabling computation to proceed while messages are in transit.

2. **Communication/computation overlap**: **Inner stencil averaging** (`avgInnerData`) and **Inner isovalue counting** (`countInnerIsoValue`) are interleaved with halo exchange, hiding a significant portion of communication latency.

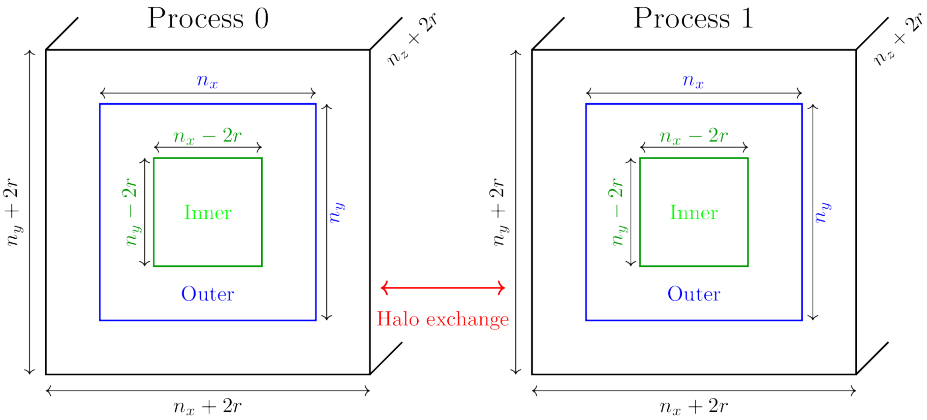

Figure 1: 2D-view of neighbouring processes, showing split of "inner" and "outer" data

3. **Custom MPI datatypes**: `MPI_Type_vector` is used for Y and Z faces, avoiding explicit packing/unpacking buffers and reducing memory copies.

4. **Field-interleaved memory layout**: Storing all fields contiguously per spatial point (`data[field + F*IDX(x,y,z)]`) improves cache locality when iterating over fields at the same point.

5. **Pointer swapping**: Instead of copying `new_data` back into `data` after each stencil step, pointers are swapped in $O(1)$.

6. **Receives posted before sends**: All `MPI_Irecv` calls are issued before any `MPI_Isend` calls to reduce the risk of message matching delays.

3 Code Compilation and Execution Instructions

Steps to Recreate our Results

```
chmod +x compile.sh
./compile.sh

sbatch job32.slurm
```

```
sbatch job48.slurm
sbatch job64.slurm
sbatch job96.slurm
```

Each SLURM script corresponds to a testcase and runs it for 5 iterations. The results and logs are stored in the `results` and `logs` folders respectively.

3.1 Compilation

The code is compiled using the shell script `compile.sh`, which loads the required MPI module and compiles using `mpicc` with `-O3`.

```
#!/bin/bash
module purge
module load compiler/oneapi-2024/mpi/2021.12
mpicc -O3 src.c -lm -o src
```

Run:

```
./compile.sh
```

3.2 Plotting

To generate plots from the results:

```
python3 plot.py
```

The script reads data from the **results** folder and generates the graphs used in the report.

4 Results

4.1 Experimental Configuration

All experiments were executed on the PARAM Rudra cluster using SLURM. The Intel MPI module `compiler/oneapi-2024/mpi/2021.12` was loaded for all runs. Each configuration was repeated 5 times.

Fixed parameters: $d = 7$, $T = 5$, `seed` $= 1000$, $F = 2$, `isovalue` $= 500$.

Table 1: Process configurations used for timing experiments

P	px	py	pz	ppn
32	4	4	2	32
48	6	4	2	48
64	4	4	4	32
96	6	4	4	48

Sub domain sizes used: $nx = ny = nz = 120$ and $nx = ny = nz = 240$.

4.2 Timing Plots

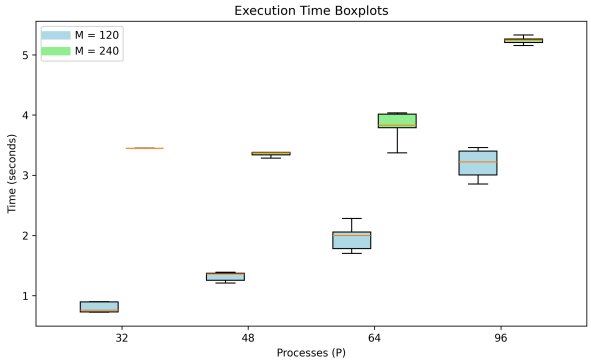

Figure 2: Figure for Time vs Number of Processes for each data size

4.3 Observations and Analysis

1. Effect of Increasing Sub domain Size: For a fixed number of processes, execution time increases when the sub domain size grows from 120^3 to 240^3 (an $8\times$ increase in data per process). This is consistent with both computation (stencil work scales as $O(nx \cdot ny \cdot nz)$) and communication (halo face sizes scale as $O(n^2)$) growing with the problem size.
2. Effect of Increasing Process Count: For both the 120^3 & 240^3 cases, increasing P increases the runtime. The observed behavior in the runtime is displayed in the above graph (see Figure 2).
3. Communication Overlap: The use of non blocking communication with overlapping inner stencil and isovalue computation is expected to reduce the effective communication cost per iteration. The benefit is much more at larger sub domain sizes where the compute-to-communicate ratio is higher.
4. Variability: Timing variability across the 5 runs is captured by the boxplots. Higher variability at larger P is expected due to network contention and synchronization overhead inherent in `MPI_Reduce` at the end of each time step.

5 Conclusions

We implemented a 3D domain decomposed MPI stencil computation with halo exchange, boundary aware averaging and isovalue counting. Key design choices include:

- Non blocking halo exchange with overlap of inner computation.
- Custom MPI vector datatypes for non contiguous face halos.
- Field-interleaved memory layout for cache efficiency.
- Separate inner/outer passes for both stencil and isovalue counting, fully hiding communication latency for sufficiently large sub-domains.

The results illustrate a tradeoff that for large sub domains the computation-to-communication ratio is favorable and scaling is efficient, while for small sub domains communication overhead limits parallel efficiency.